LLMы

Хотим

На берегу знать, как оптимально обучить LLM за бюджет

- Размер модели
- Размер датасета
- Количество шагов обучения (сколько токенов модель увидит)
- Batch size, lr, depth, width, ...

Scaling Laws

зависимость лосса от размера модели, данных, компьюта

GPT-3 вышел через 3 месяца

Scaling Laws for Neural Language Models

Jared Kaplan *
Johns Hopkins University, OpenAI
jaredk@jhu.edu

Sam McCandlish*
OpenAI
sam@openai.com

Tom Henighan
OpenAI
henighan@openai.com

Tom B. Brown
OpenAI
tom@openai.com

Benjamin Chess
OpenAI
bchess@openai.com

Rewon Child
OpenAI
rewon@openai.com

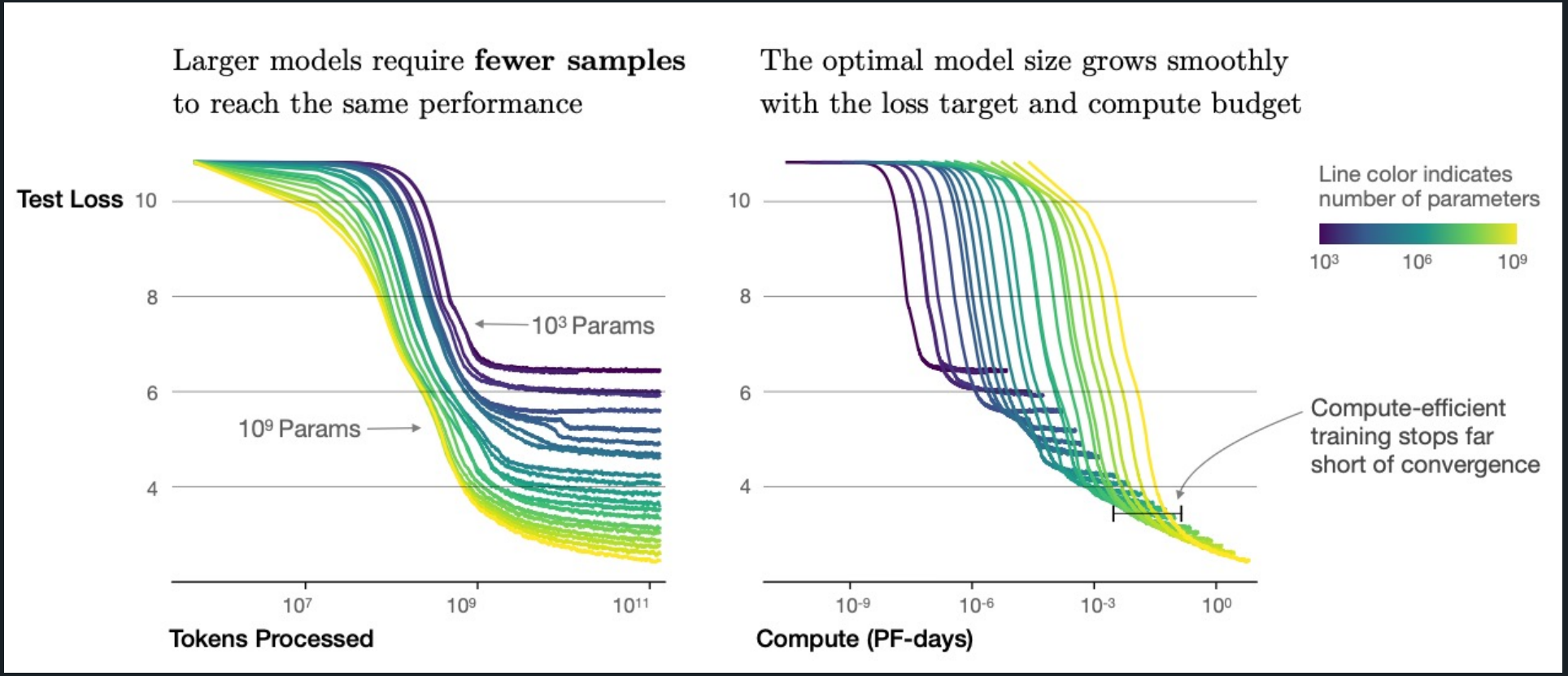
Scott Gray
OpenAI
scott@openai.com

Alec Radford
OpenAI
alec@openai.com

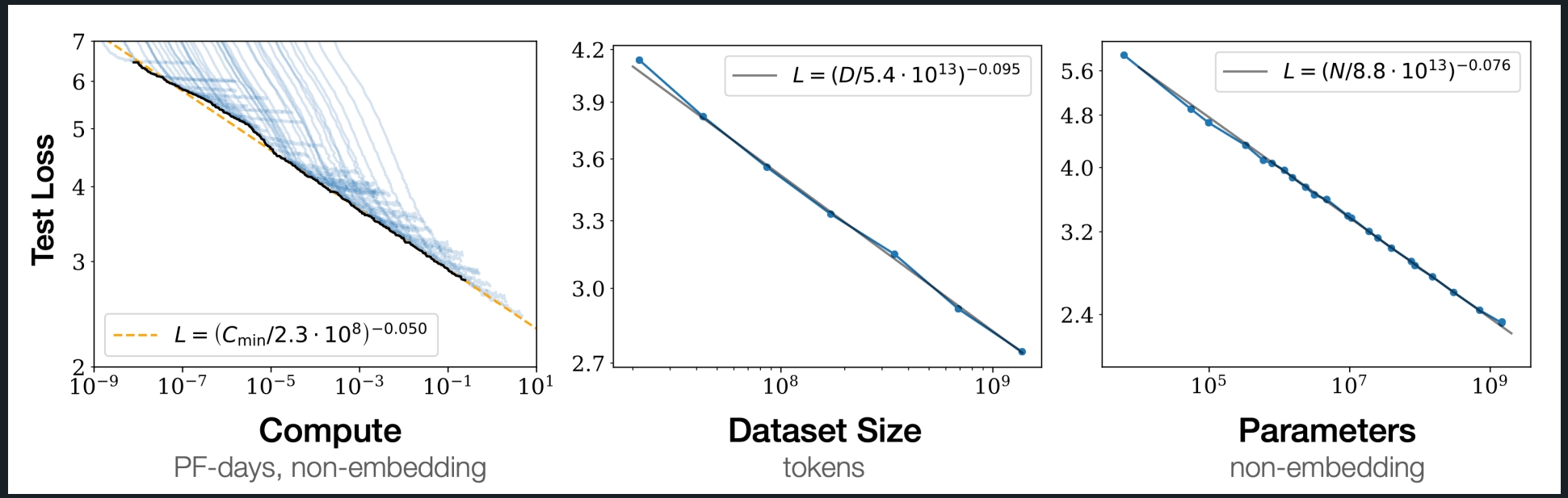
Jeffrey Wu
OpenAI
jeffwu@openai.com

Dario Amodei
OpenAI
damodei@openai.com

Scaling Laws – compute efficient



Scaling Laws – predictive power law



N – кол-во параметров

D = B * S – кол-во данных

C – compute

Лосс зависит от них экспоненциально, а от глубина и ширины – не зависит

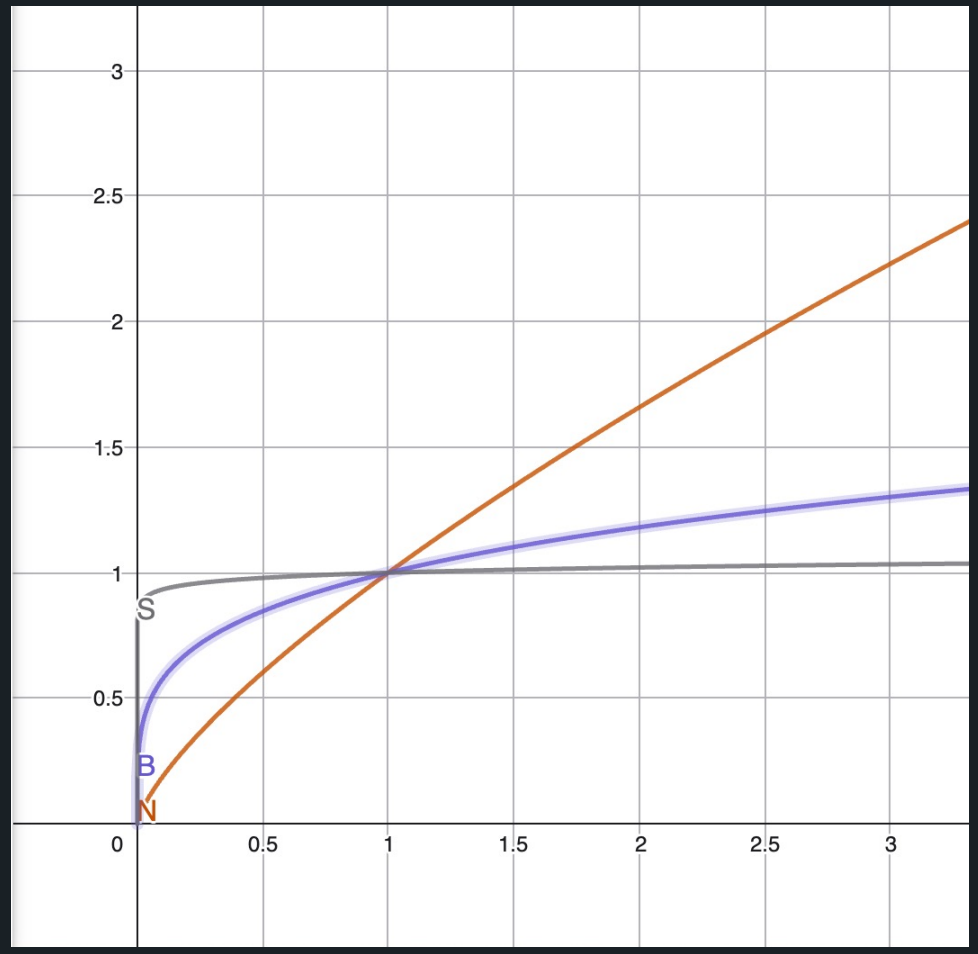
Scaling Laws – optimal for C scaling laws

for fixed C :

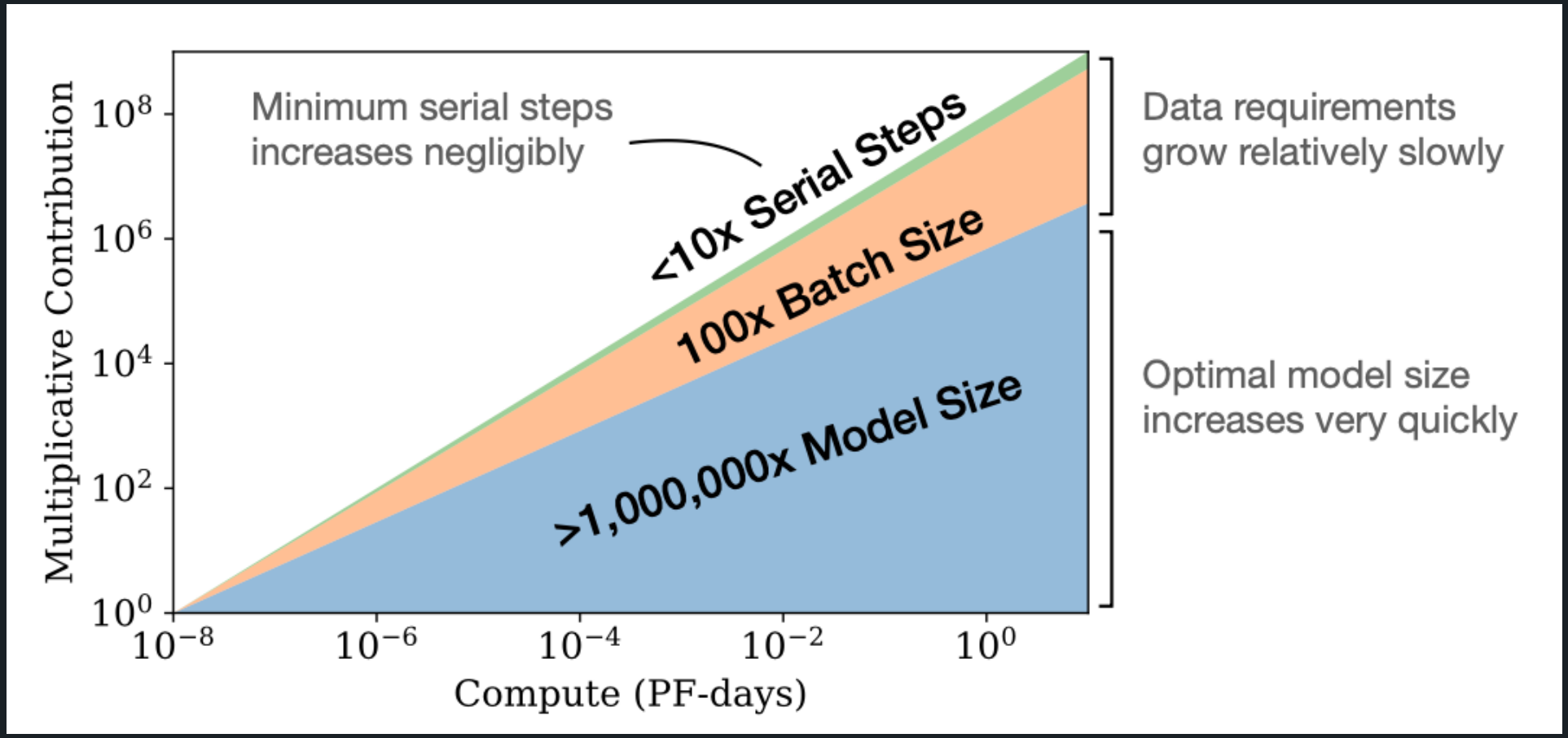
$$N \propto C^{\frac{\alpha_C^{\min}}{\alpha_N}}; \quad B \propto C^{\frac{\alpha_C^{\min}}{\alpha_B}}; \quad S \propto C^{\frac{\alpha_C^{\min}}{\alpha_S}},$$

$$\alpha_C^{\min} = \frac{1}{\frac{1}{\alpha_S} + \frac{1}{\alpha_B} + \frac{1}{\alpha_N}}$$

$$N \propto C^{0.73}, \quad B \propto C^{0.24}, \quad S \propto C^{0.03}$$



Scaling Laws – optimal scaling



Scaling Laws – optimal batch size

$$B_{\text{crit}}(L) = \frac{B_*}{L^{\frac{1}{\alpha_B}}}, B_* \sim 1 * 10^8 \text{ tokens}, \alpha_B \sim 0.21$$

Scaling Laws – вывод

- Power laws
- Лучше не долго обучать большую модель, чем долго маленьку
- Сильнее всего надо скейлить количество параметров

Performance depends strongly on scale, weakly on model shape: Model performance depends most strongly on scale, which consists of three factors: the number of model parameters N (excluding embeddings), the size of the dataset D, and the amount of compute C used for training. Within reasonable limits, performance depends very weakly on other architectural hyperparameters such as depth vs. width. (Section 3)

Smooth power laws: Performance has a power-law relationship with each of the three scale factors N, D, C when not bottlenecked by the other two, with trends spanning more than six orders of magnitude (see Figure 1). We observe no signs of deviation from these trends on the upper end, though performance must flatten out eventually before reaching zero loss. (Section 3)

Universality of overfitting: Performance improves predictably as long as we scale up N and D in tandem, but enters a regime of diminishing returns if either N or D is held fixed while the other increases. The performance penalty depends predictably on the ratio $N^{0.74}/D$, meaning that every time we increase the model size 8x, we only need to increase the data by roughly 5x to avoid a penalty. (Section 4)

Universality of training: Training curves follow predictable power-laws whose parameters are roughly independent of the model size. By extrapolating the early part of a training curve, we can roughly predict the loss that would be achieved if we trained for much longer. (Section 5)

Transfer improves with test performance: When we evaluate models on text with a different distribution than they were trained on, the results are strongly correlated to those on the training validation set with a roughly constant offset in the loss – in other words, transfer to a different distribution incurs a constant penalty but otherwise improves roughly in line with performance on the training set. (Section 3.2.2)

Sample efficiency: Large models are more sample-efficient than small models, reaching the same level of performance with fewer optimization steps (Figure 2) and using fewer data points (Figure 4).

Convergence is inefficient: When working within a fixed compute budget C but without any other restrictions on the model size N or available data D, we attain optimal performance by training *very large models* and stopping *significantly short of convergence* (see Figure 3). Maximally compute-efficient training would therefore be far more sample efficient than one might expect based on training small models to convergence, with data requirements growing very slowly as $D \sim C^{0.27}$ with training compute. (Section 6)

Optimal batch size: The ideal batch size for training these models is roughly a power of the loss only, and continues to be determinable by measuring the gradient noise scale [MKAT18]; it is roughly 1-2 million tokens at convergence for the largest models we can train. (Section 5.1)

Compute-Optimal Models

DeepMind

Training Compute-Optimal Large Language Models

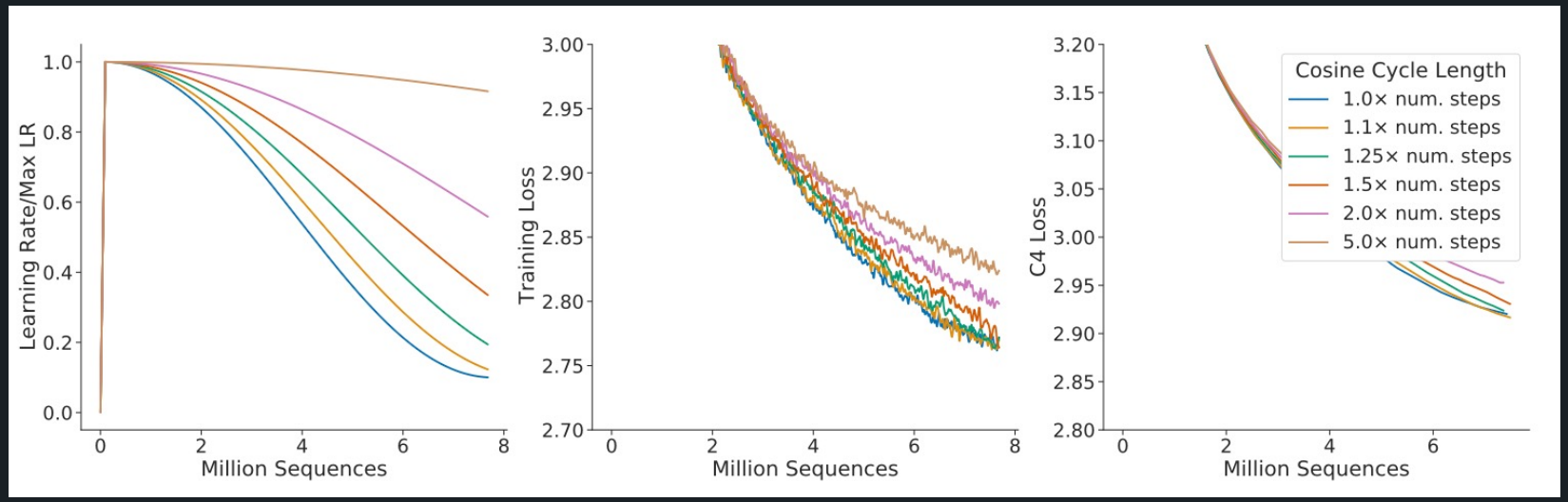
Jordan Hoffmann*, Sebastian Borgeaud*, Arthur Mensch*, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals and Laurent Sifre*

*Equal contributions

We investigate the optimal model size and number of tokens for training a transformer language model under a given compute budget. We find that current large language models are significantly under-trained, a consequence of the recent focus on scaling language models whilst keeping the amount of training data constant. By training over 400 language models ranging from 70 million to over 16 billion parameters on 5 to 500 billion tokens, we find that for compute-optimal training, the model size and the number of training tokens should be scaled equally: for every doubling of model size the number of training tokens should also be doubled. We test this hypothesis by training a predicted compute-optimal model, *Chinchilla*, that uses the same compute budget as *Gopher* but with 70B parameters and 4× more more data. *Chinchilla* uniformly and significantly outperforms *Gopher* (280B), GPT-3 (175B), Jurassic-1 (178B), and Megatron-Turing NLG (530B) on a large range of downstream evaluation tasks. This also means that *Chinchilla* uses substantially less compute for fine-tuning and inference, greatly

29 Mar 2022

Compute-Optimal Models

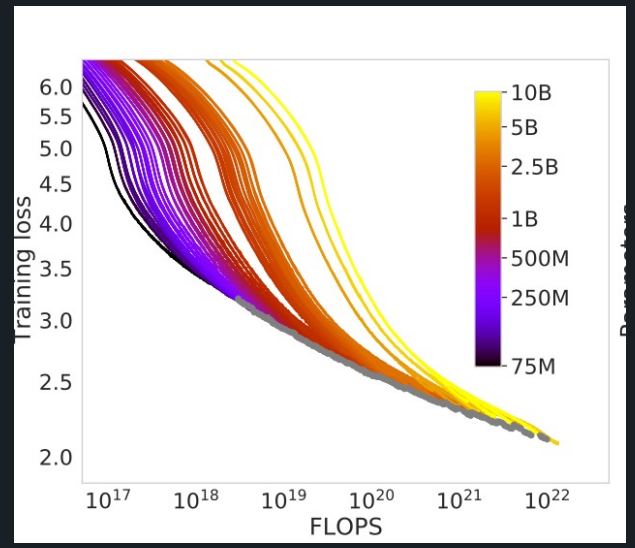


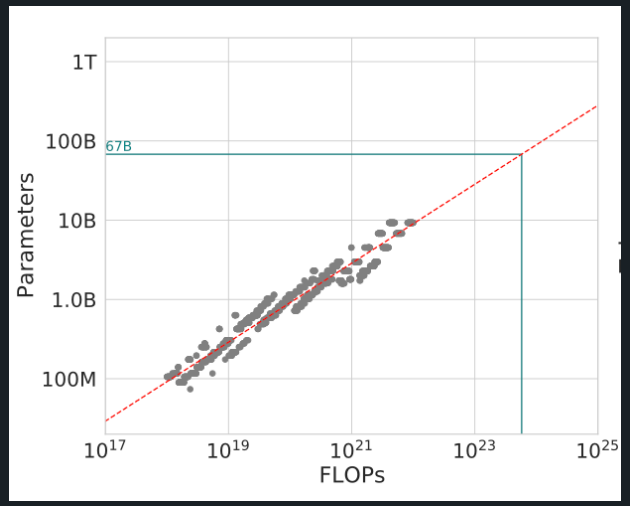
Compute-Optimal Models – Approach 1

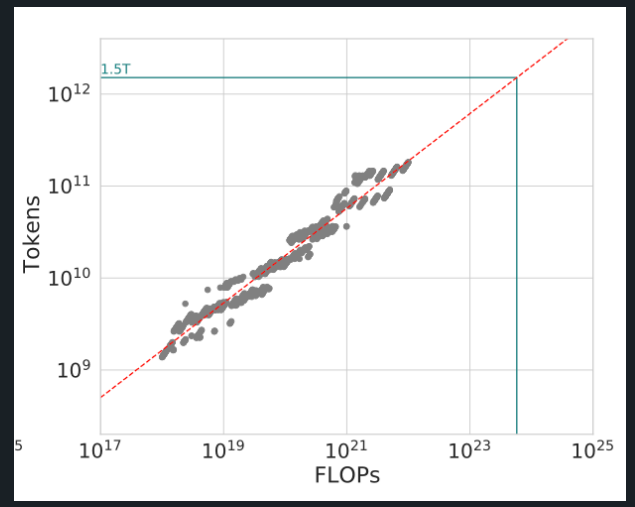
$$\forall N \in \{70M, ..., 10B\}, S \in \{s_1, s_2, s_3, s_4\} : \text{Loss}(N, S).$$

Для заданного набора размеров модели и продолжительностей обучения посчитали лосс

$$\forall C : N_{\text{opt}}, S_{\text{opt}} = \underset{N,S:\ \text{FLOPS}(N,S)=C}{\text{argmin}} \text{Loss}(N, S)$$



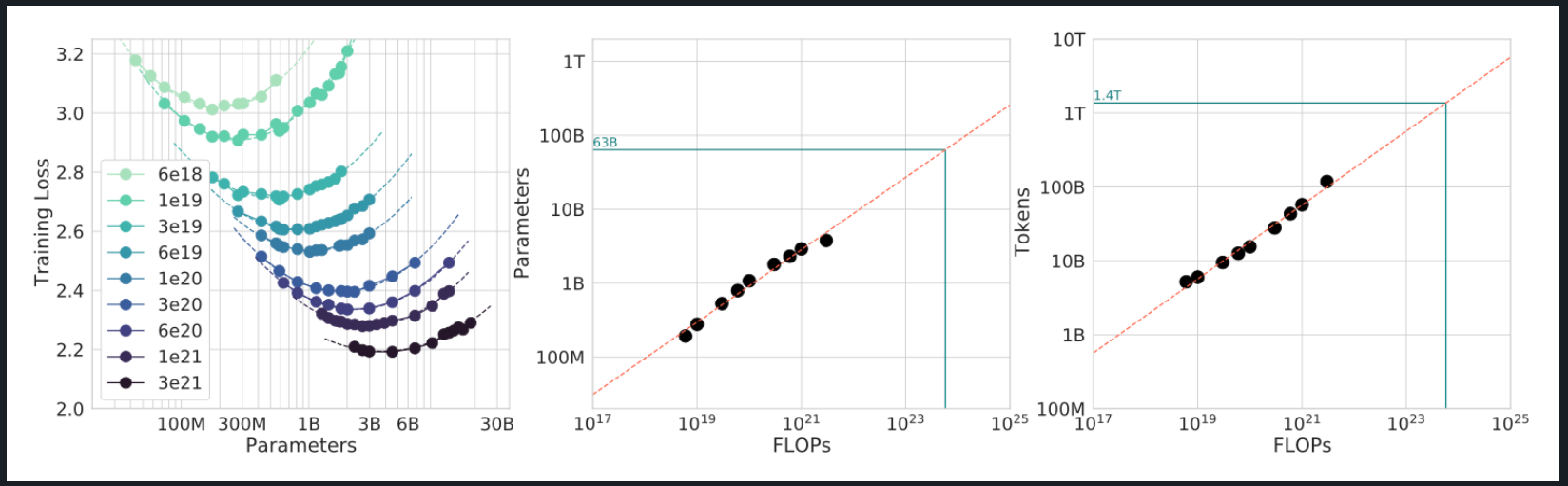




Compute-Optimal Models – Approach 2

$$\forall N \in \{70M, ..., 10B\}, C \in \{c_1, ..., c_9\}, S : \text{FLOPS}(N, S) = C : \text{Loss}(N, S).$$

Для фиксированного Flops бюджета варьируем соотношения размеров модели и продолжительности обучения

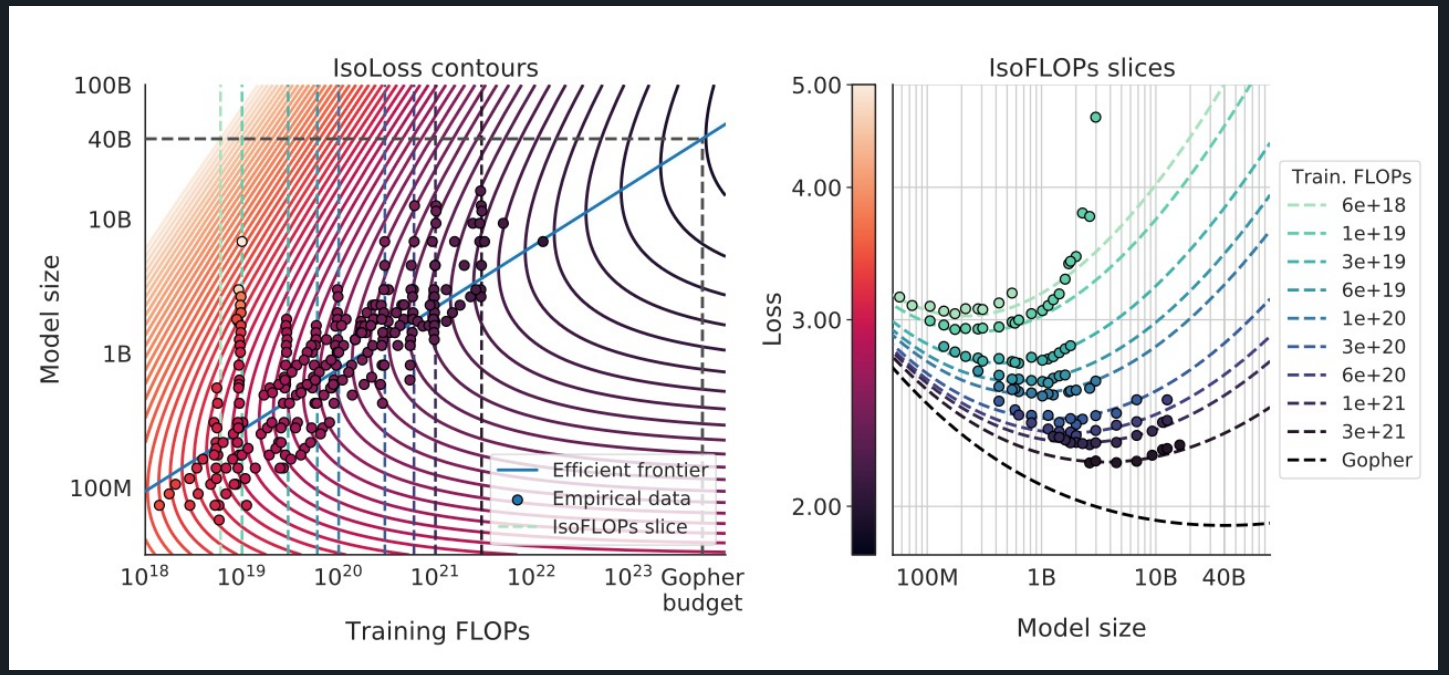


Compute-Optimal Models – Approach 3

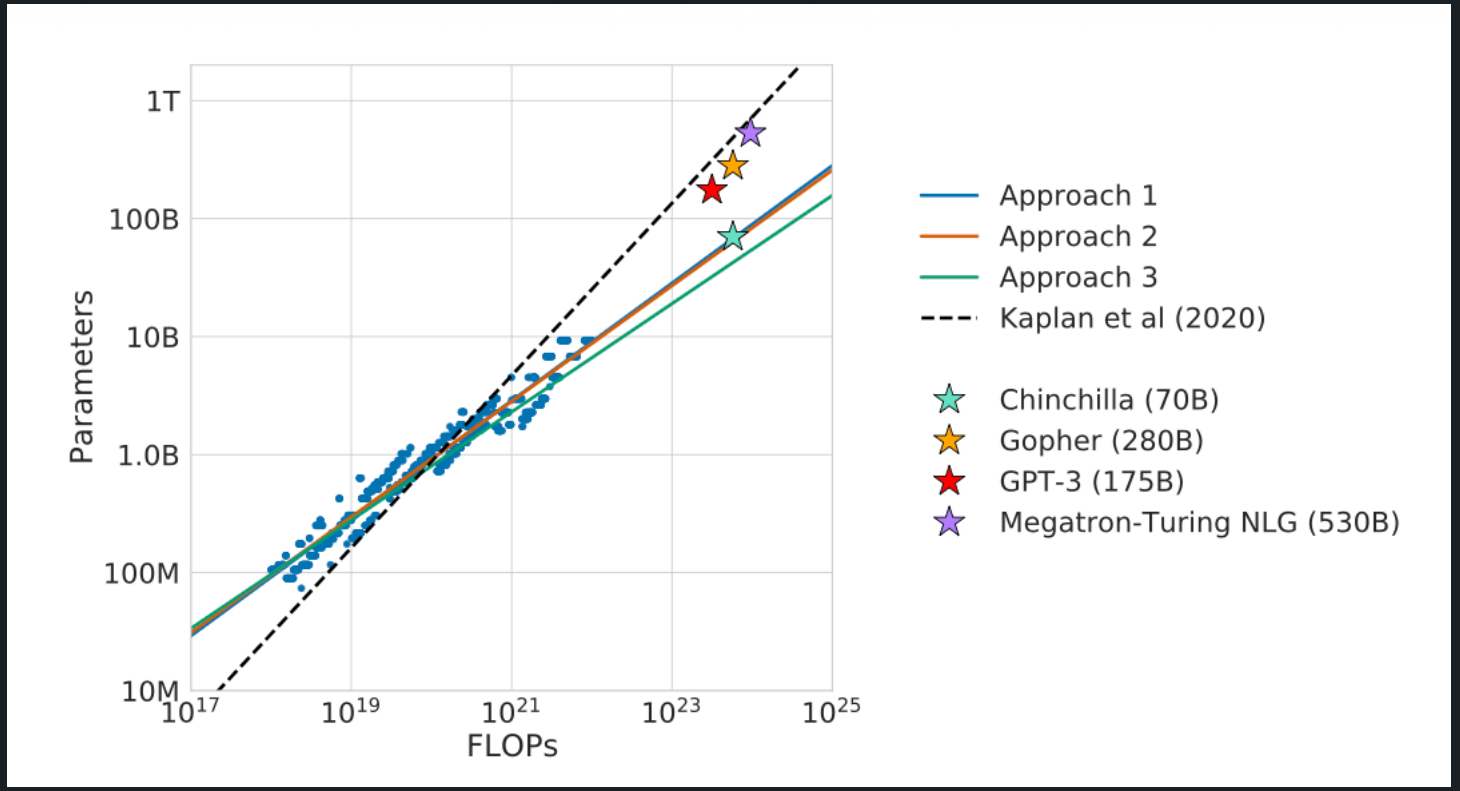
На метриках всех эксперементах построили функцию оценки лосса :

$$\hat{L}(N, D) = \underbrace{E}_{\substack{\text{Идеальный лосс} \\ \text{a.k. энтропия текста}}} + \overbrace{\frac{A}{N^{\alpha}}}^{\substack{\text{ошибка от} \\ \text{неитеальности} \\ \text{архитектуры}}} + \underbrace{\frac{B}{D^{\beta}}}_{\substack{\text{ошибка от} \\ \text{неидеальности} \\ \text{и конечности} \\ \text{процесса обучения}}}$$

обучили A, B, E, α, β на метриках своих эксперементов



Compute-Optimal Models



Compute-Optimal Models

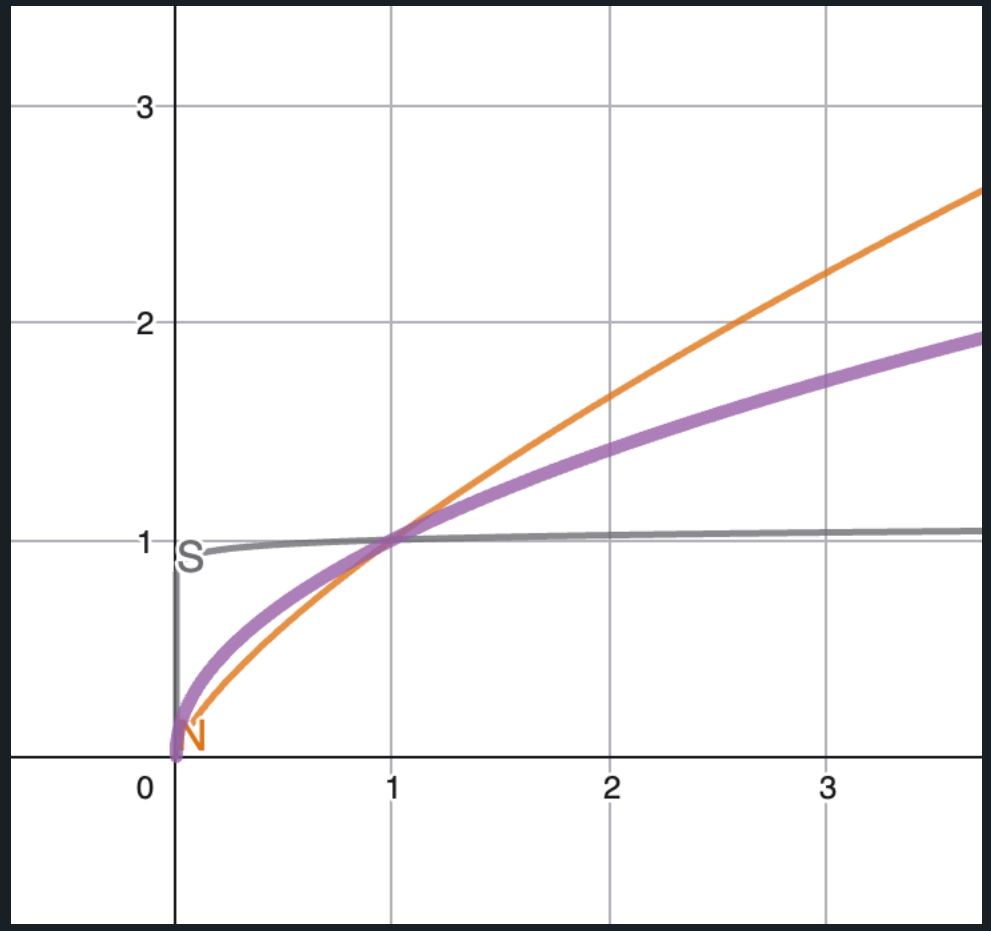
for fixed C :

$$N \propto C^{\frac{\alpha_C^{\min}}{\alpha_N}}; \quad B \propto C^{\frac{\alpha_C^{\min}}{\alpha_B}}; \quad S \propto C^{\frac{\alpha_C^{\min}}{\alpha_S}},$$

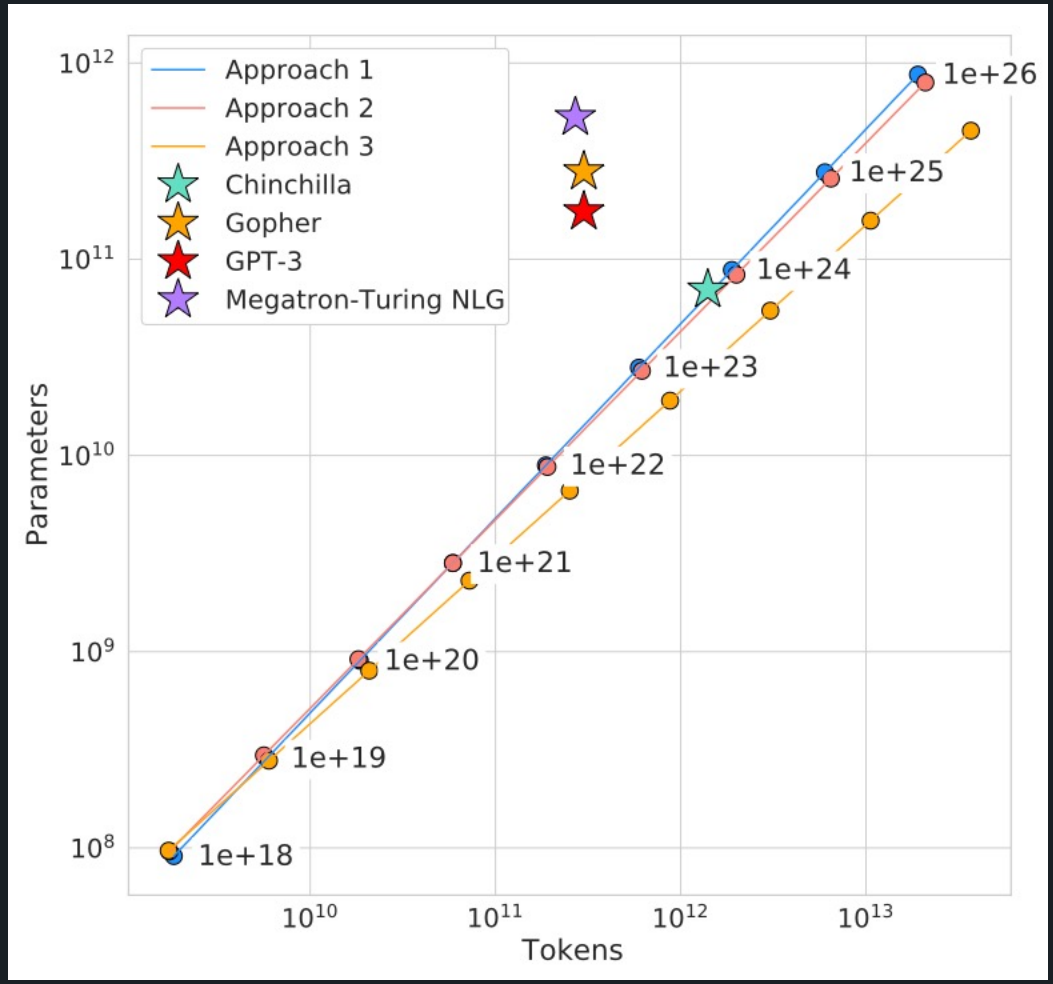
$$\alpha_C^{\min} = \frac{1}{\frac{1}{\alpha_S} + \frac{1}{\alpha_B} + \frac{1}{\alpha_N}}$$

$$N \propto C^{0.73}, \quad B \propto C^{0.24}, \quad S \propto C^{0.03}$$

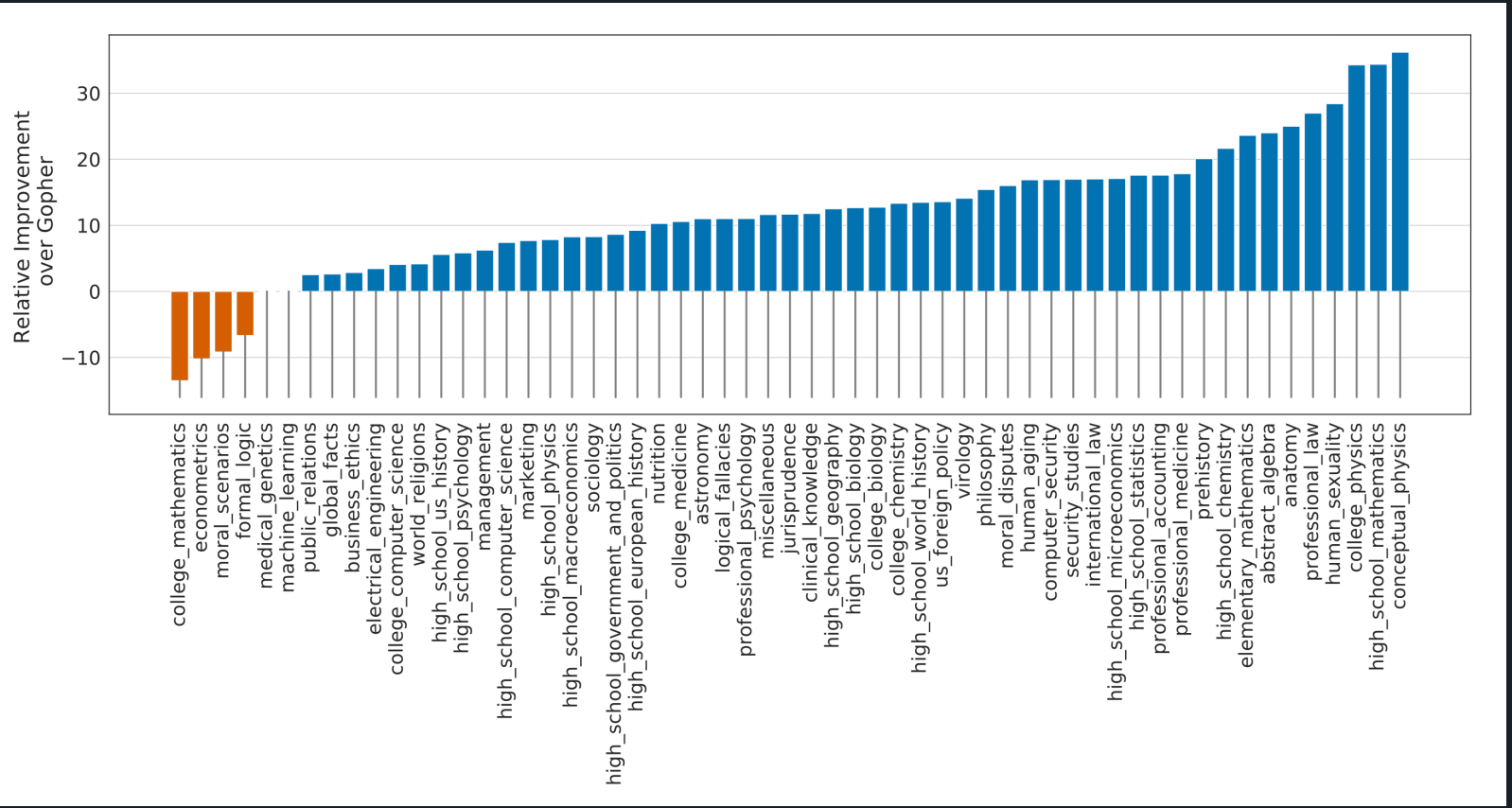
Approach	Coeff. a where $N_{opt} \propto C^a$	Coeff. b where $D_{opt} \propto C^b$
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric modelling of the loss	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan et al. (2020)	0.73	0.27



Compute-Optimal Models



Compute-Optimal Models



Compute-Optimal Models

Random	25.0%
Average human rater	34.5%
GPT-3 5-shot	43.9%
Gopher 5-shot	60.0%
***Chinchilla* 5-shot**	**67.6%**
Average human expert performance	*89.8%*
June 2022 Forecast	57.1%
June 2023 Forecast	63.4%